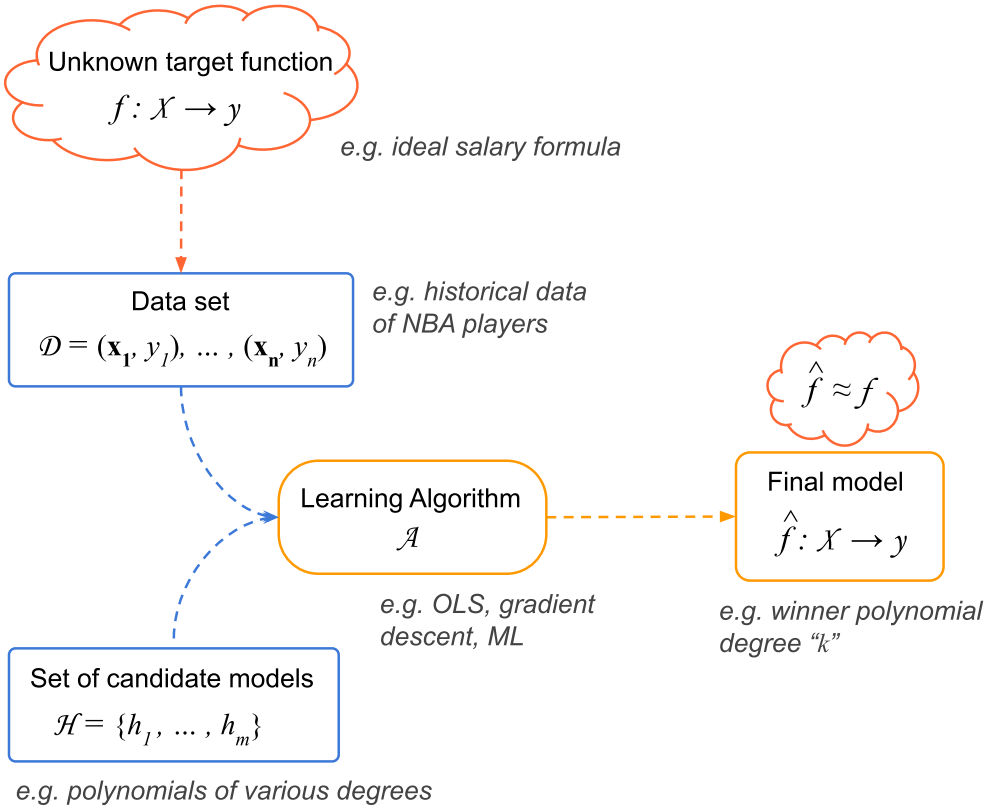
Learning Phases

Ngoc Hoang Luong

University of Information Technology (UIT), VNU-HCM

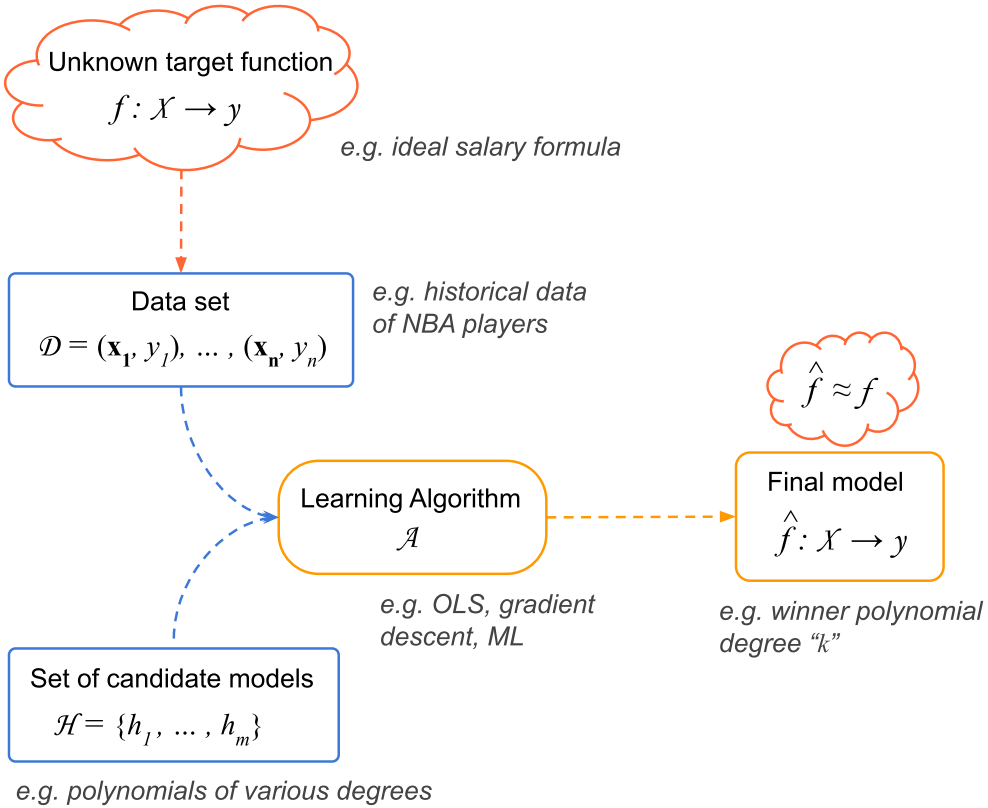
October 9, 2026

Review: Theoretical Framework for Supervised Learning



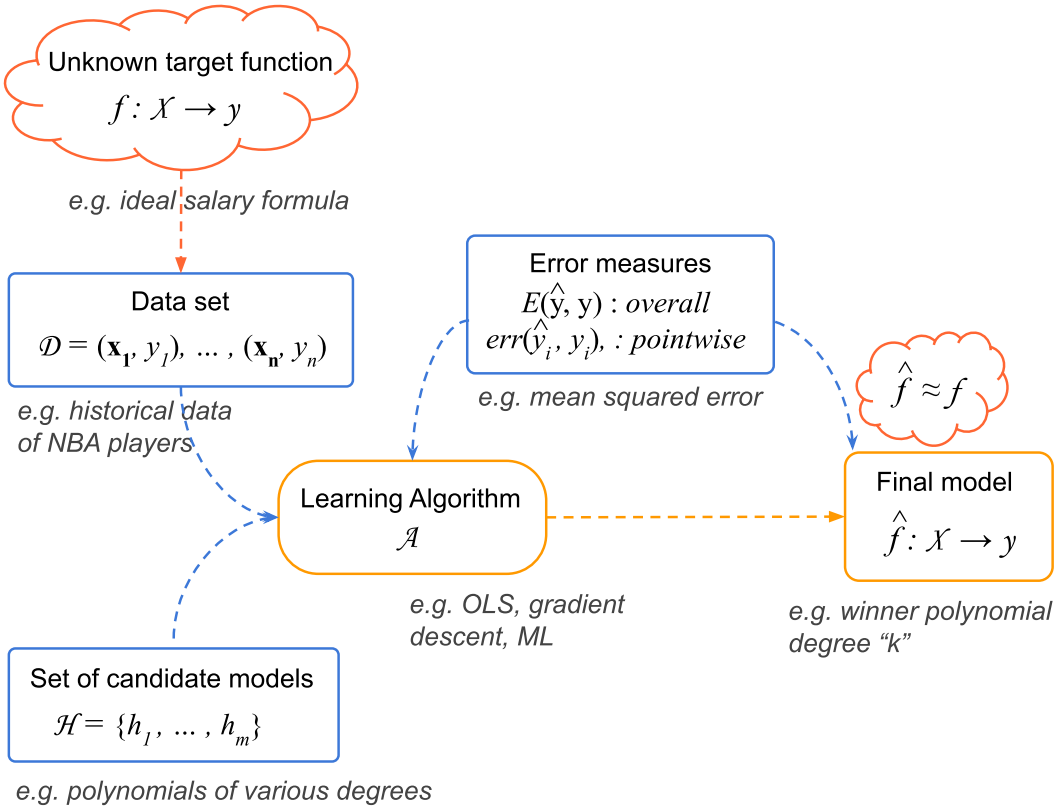
- The target function f is unknown. From the observed data, we can only learn an approximation $\hat{f}$ to f.
- Since f itself is unknown, we cannot directly determine how close $\hat{f}$ is to f.

Review: Theoretical Framework for Supervised Learning



- We observe a dataset $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$, influenced by the unknown target function.
- The hypothesis set $\mathcal{H}$ contains the candidate prediction functions that we allow the learning algorithm to consider, e.g., linear models, polynomial models, or non-parametric models.

Review: Theoretical Framework for Supervised Learning



- The learning algorithm $\mathcal{A}$ uses $\mathcal{D}$ to select a hypothesis $h \in \mathcal{H}$.
- A loss function $err(\hat{y}, y)$ measures the error of an individual prediction. An overall error measure $E(h)$ summarizes the losses over multiple observations and is used to evaluate the predictive performance of h.

Review: Theoretical Framework for Supervised Learning

- In ML, the error of an individual prediction is measured by a **loss function**:
  - Squared error: $err(\hat{f}(\mathbf{x}_i), y_i) = (\hat{y}_i - y_i)^2$
  - Absolute error: $err(\hat{f}(\mathbf{x}_i), y_i) = |\hat{y}_i - y_i|$
  - Misclassification error: $err(\hat{f}(\mathbf{x}_i), y_i) = \mathbf{1}[\hat{y}_i \neq y_i]$
- The **in-sample error** E_{in}, or empirical risk, is the average loss over the observed data:

$$E_{in}(\hat{f}) = \frac{1}{|\mathcal{D}_{in}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{in}} err(\hat{f}(\mathbf{x}_i), y_i).$$

- The **out-of-sample error** E_{out}, or population risk, is the expected loss over the data-generating distribution:

$$E_{out}(\hat{f}) = \mathbb{E}_{(\mathbf{x},y)\sim P}\left[err(\hat{f}(\mathbf{x}), y)\right].$$

- Here, $\mathbf{x} \in \mathcal{X}$ is an input and $y \in \mathcal{Y}$ is its corresponding target.

Review: Theoretical Framework for Supervised Learning

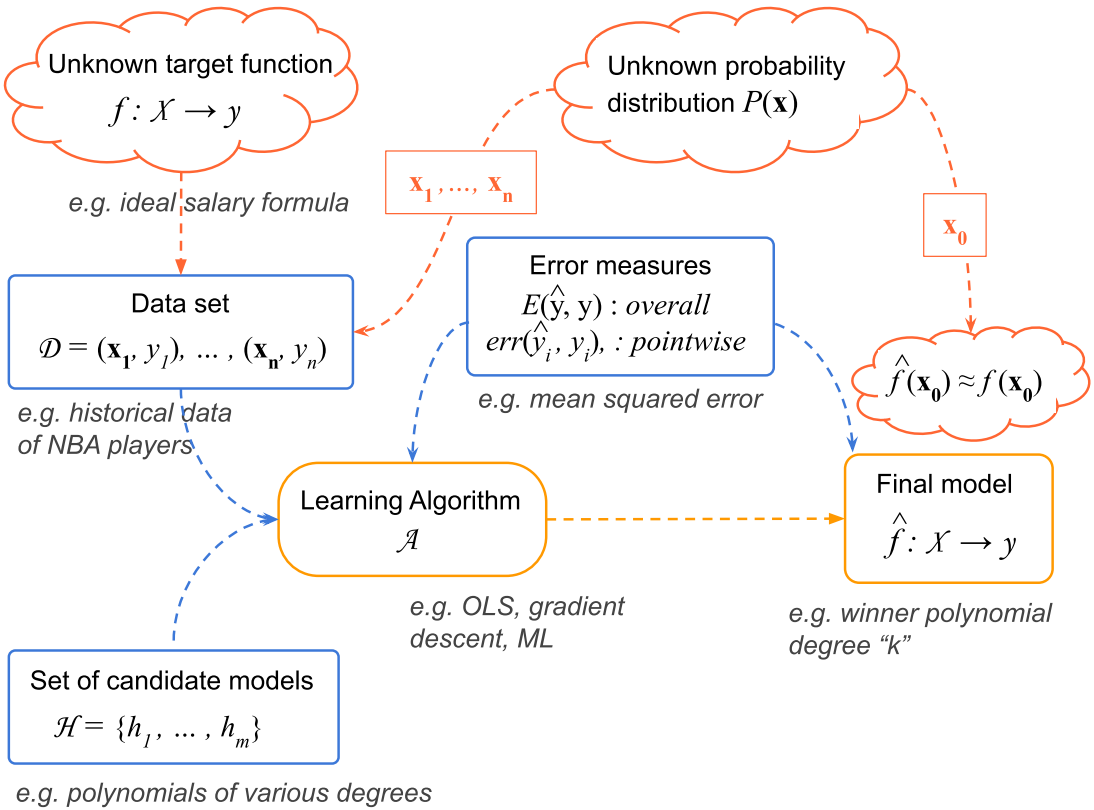
- Our goal is to find a model $\hat{f}$ with a small **out-of-sample error** $E_{out}(\hat{f})$.
- However, we cannot compute E_{out} exactly because the underlying data-generating distribution P is unknown.
- We approach this problem through two goals:

$$E_{out}(\hat{f}) \text{ small} \Leftarrow \begin{cases} E_{in}(\hat{f}) \text{ small} & \text{practical goal} \\ E_{out}(\hat{f}) \approx E_{in}(\hat{f}) & \text{generalization goal} \end{cases}$$

- The first goal can be addressed because we have access to the training data $\mathcal{D}_{in}$.
- For the second goal, we assume that the observed data points are independent and identically distributed (i.i.d.) samples from the data-generating distribution:

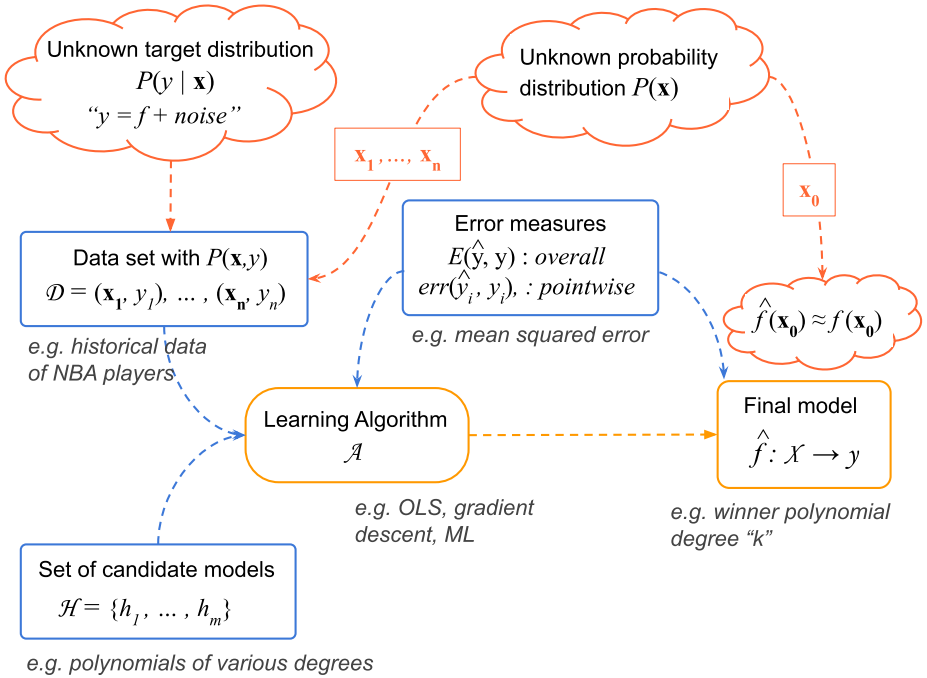
$$(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n) \overset{\text{i.i.d.}}{\sim} P.$$

Review: Theoretical Framework for Supervised Learning



$$E_{out}(\hat{f}) \text{ small} \Leftarrow \begin{cases} E_{in}(\hat{f}) \text{ small} & \text{practical goal} \\ E_{out}(\hat{f}) \approx E_{in}(\hat{f}) & \text{generalization goal} \end{cases}$$

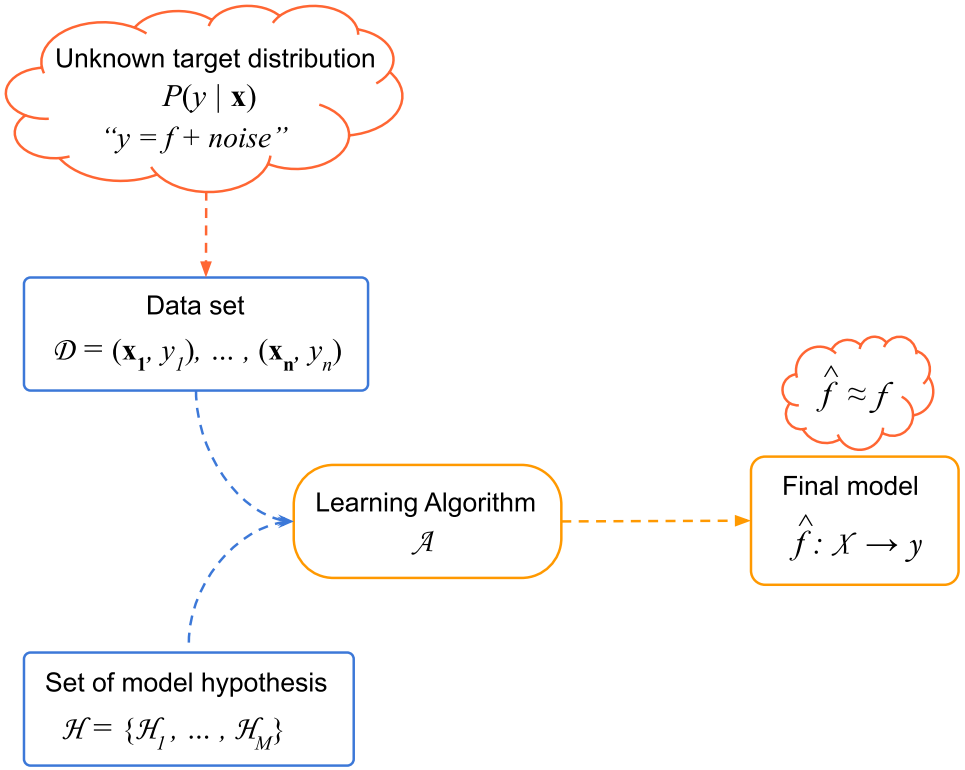
Review: Theoretical Framework for Supervised Learning



- In practice, the target may be noisy: $y = f(\mathbf{x}) + \varepsilon$.
- Therefore, the output y is not necessarily determined uniquely by $\mathbf{x}$. It is characterized by a **conditional distribution** $P(y \mid \mathbf{x})$.
- The data-generating distribution P is the joint distribution of inputs and outputs:

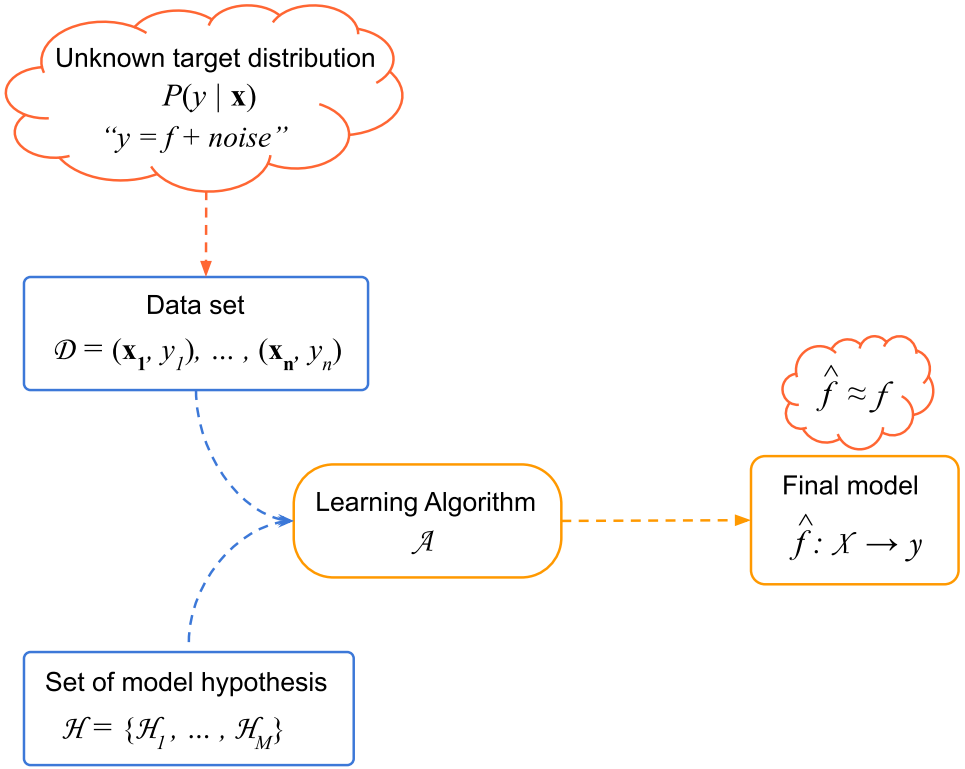
$$P(\mathbf{x}, y) = P(\mathbf{x})P(y \mid \mathbf{x}).$$

Learning Phases



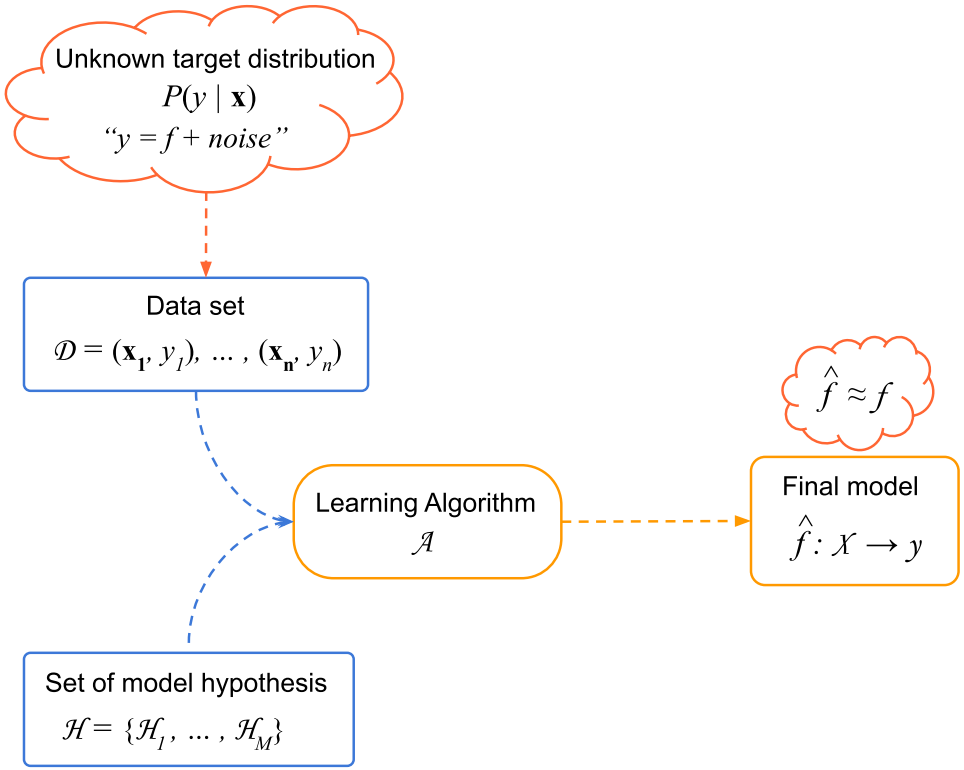
- We often consider different **hypothesis classes**. For example: principal component regression $\mathcal{H}_1$, ridge regression $\mathcal{H}_2$, and LASSO $\mathcal{H}_3$.
- Each hypothesis class may involve one or more **hyperparameters** whose values need to be selected.

Learning Phases



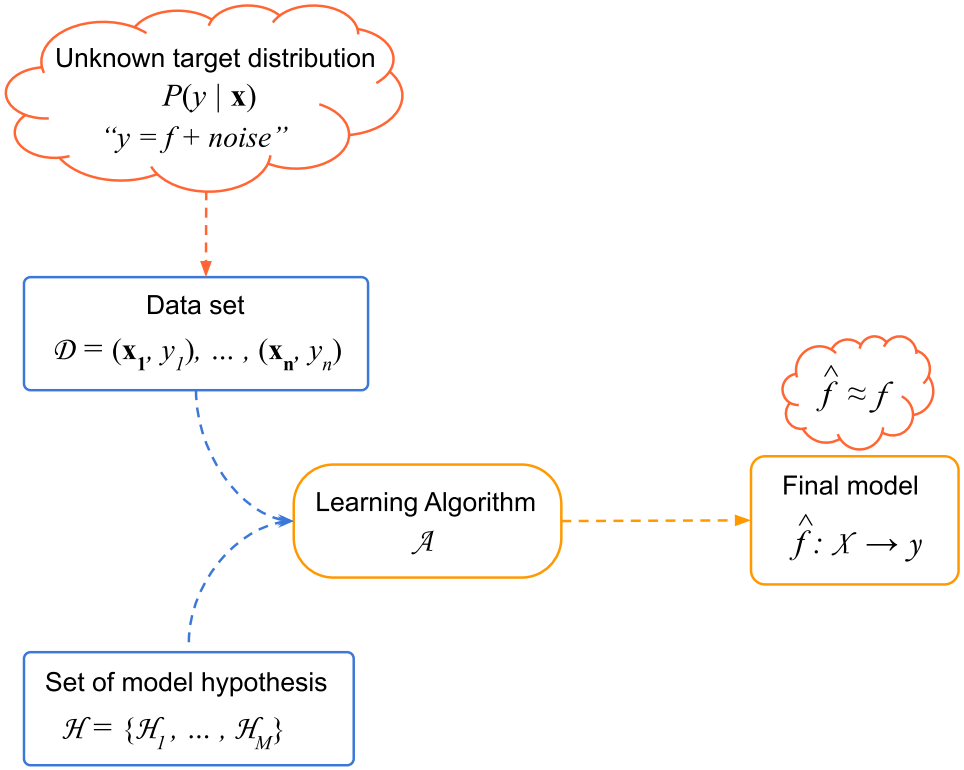
- Within each hypothesis class, we need to select suitable hyperparameter values and obtain a **finalist model**.
- Among the finalist models from different hypothesis classes, we then select the **final model** $\hat{f}$.

Learning Phases



- Finally, we need to assess the **predictive performance** of the final model $\hat{f}$ on unseen data.
- In particular, we want to estimate its out-of-sample error $E_{out}(\hat{f})$.

Learning Phases



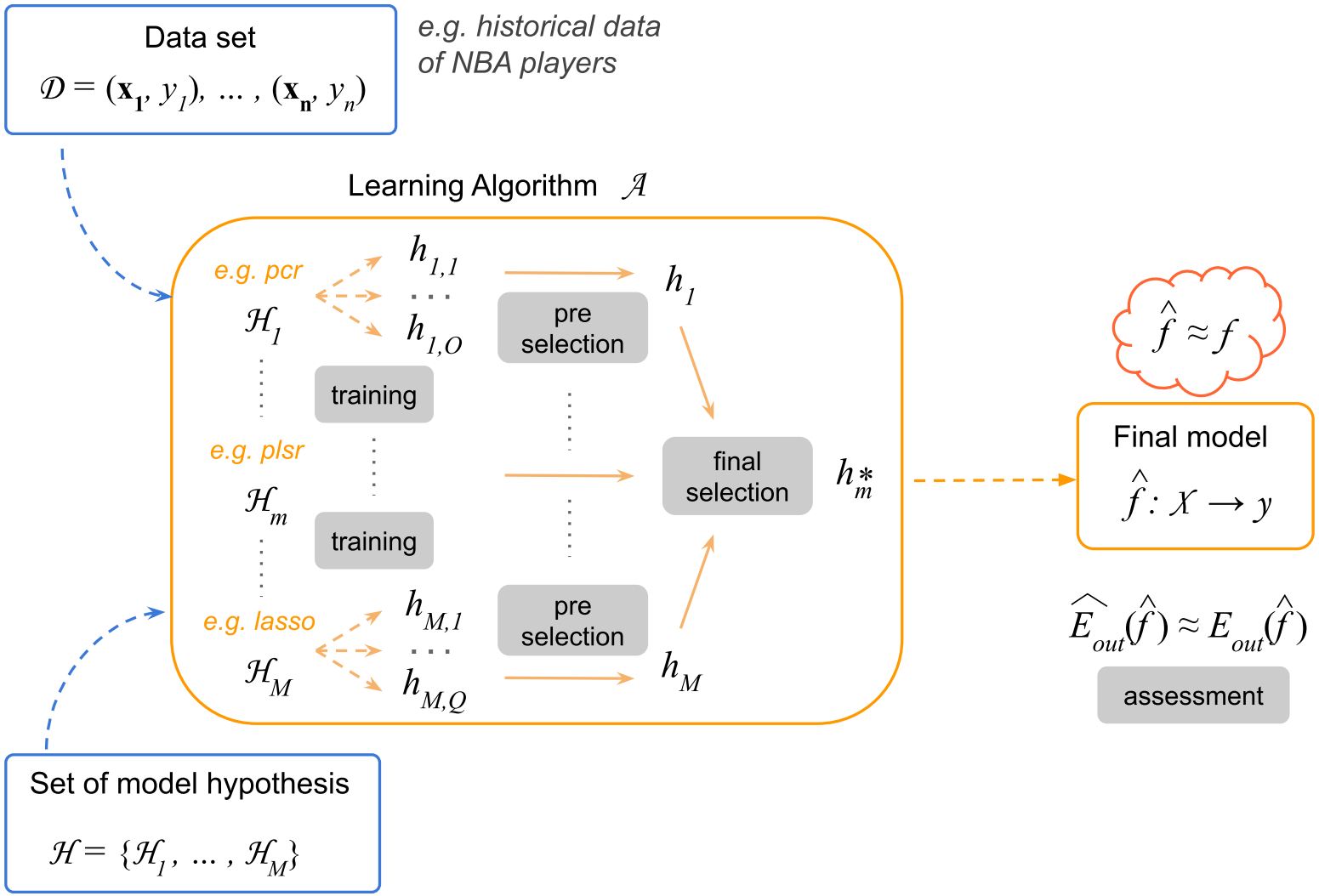
The supervised learning process can be organized into three phases:

1. Model training
2. Model selection
3. Model assessment

Learning Phases

1. Training: For each hypothesis class $\mathcal{H}_1, \ldots, \mathcal{H}_M$, we fit candidate models using the available training data.
2. Selection: We choose models based on their estimated predictive performance.
   - **Pre-Selection**: Choose a finalist model within each hypothesis class, e.g., by selecting its hyperparameters.
   - **Final-Selection**: Choose the final model from the set of finalist models.
3. Assessment: Given the final model $\hat{f}$, estimate its out-of-sample error $E_{out}(\hat{f})$.

Learning Phases



What data should we use for training, selection, and assessment?

Model Assessment

- Given a final model h, how can we measure its predictive performance?
- The in-sample error E_{in} measures how well the model fits the data used for training:

$$E_{in}(h) = \frac{1}{|\mathcal{D}_{in}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{in}} err(h(\mathbf{x}_i), y_i).$$

- The out-of-sample error E_{out} measures the **generalization performance** of the model:

$$E_{out}(h) = \mathbb{E}_{(\mathbf{x},y)\sim P} \left[err(h(\mathbf{x}), y) \right].$$

- Because E_{out} is a population quantity and cannot be computed exactly, we estimate it using an independent sample $\mathcal{D}_{proxy}$ drawn from the same data-generating distribution:

$$\hat{E}_{out}(h) = E_{proxy}(h) \approx E_{out}(h).$$

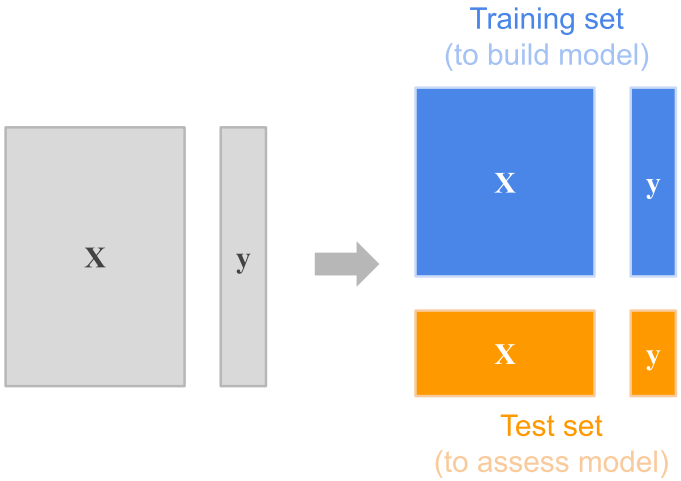
Model Assessment - Holdout Test Set

- How can we estimate E_{out}?
- Suppose the available dataset is

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}.$$

  If we use all n observations to fit the model, we can compute E_{in}, but we have no independent data left for an **honest** estimate of E_{out}.
- Why not use E_{in} as an estimate of E_{out}?
- A model may have a very small in-sample error E_{in} but still perform poorly on unseen data.
- The Holdout Method splits the available dataset $\mathcal{D}$ into two disjoint subsets:
  1. **Training set** $\mathcal{D}_{train}$: used to fit the model.
  2. **Test set** $\mathcal{D}_{test}$: kept independent of model fitting and used to estimate the out-of-sample error E_{out}.

Model Assessment - Holdout Test Set

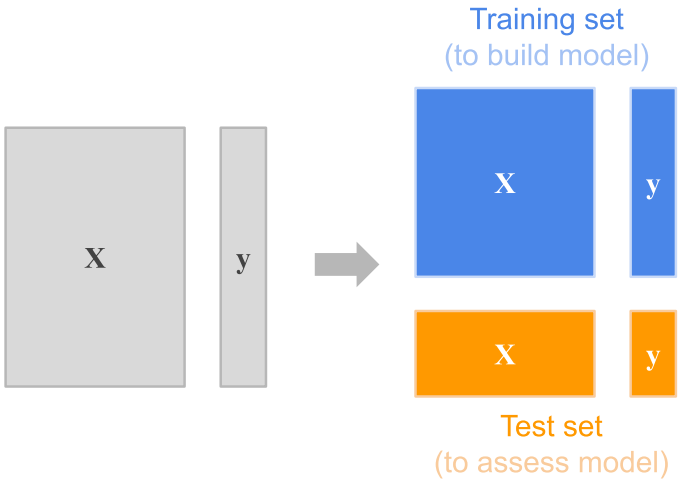


Randomly select a observations without replacement from $\mathcal{D}$ to form the test set:

$$\mathcal{D} \to \begin{cases} \mathcal{D}_{train} & \text{size } n-a, \\ \mathcal{D}_{test} & \text{size } a. \end{cases}$$

Fit a model on $\mathcal{D}_{train}$ to obtain h^-.

Model Assessment - Holdout Test Set



Randomly select a observations without replacement from $\mathcal{D}$ to form the test set:

$$\mathcal{D} \rightarrow \begin{cases} \mathcal{D}_{train} & \text{size } n-a, \\ \mathcal{D}_{test} & \text{size } a. \end{cases}$$

Using the held-out observations in $\mathcal{D}_{test}$, we estimate the out-of-sample performance of h^- by:

$$E_{test}(h^-) = \frac{1}{a} \sum_{(\mathbf{x}_l, y_l) \in \mathcal{D}_{test}} err(h^-(\mathbf{x}_l), y_l).$$

Model Assessment - Holdout Test Set

- Suppose $\mathcal{D}_{test}$ consists of observations drawn from the same data-generating distribution P and is independent of the fitting of h^-. Can it provide an unbiased estimate of $E_{out}(h^-)$?
- Consider one held-out observation $(\mathbf{x}_0, y_0) \sim P$. Its prediction error is:

$$err(h^-(\mathbf{x}_0), y_0).$$

- Treating the trained model h^- as fixed, its expected prediction error is:

$$\mathbb{E}_{(\mathbf{x}_0,y_0)\sim P}\left[err(h^-(\mathbf{x}_0), y_0)\right].$$

- By definition, this expectation is the out-of-sample error:

$$\mathbb{E}_{(\mathbf{x}_0,y_0)\sim P}\left[err(h^-(\mathbf{x}_0), y_0)\right] = E_{out}(h^-).$$

- Therefore, the error on one independent test observation is an **unbiased estimator** of $E_{out}(h^-)$.

Model Assessment - Holdout Test Set - Expectation

- Let $\mathcal{D}_{test} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_a, y_a)\}$ contain a independent test observations. Then

$$E_{test}(h^-) = \frac{1}{a}\sum_{l=1}^{a} err(h^-(\mathbf{x}_l), y_l).$$

- Treating the trained model h^- as fixed, is $E_{test}(h^-)$ an unbiased estimate of $E_{out}(h^-)$?

$$\begin{aligned}\mathbb{E}[E_{test}(h^-)] &= \mathbb{E}\left[\frac{1}{a}\sum_{l=1}^{a} err(h^-(\mathbf{x}_l), y_l)\right] \\ &= \frac{1}{a}\sum_{l=1}^{a} \mathbb{E}_{(\mathbf{x}_l, y_l)\sim P}\left[err(h^-(\mathbf{x}_l), y_l)\right] \\ &= \frac{1}{a}\sum_{l=1}^{a} E_{out}(h^-) = E_{out}(h^-).\end{aligned}$$

- Therefore, $E_{test}(h^-)$ is an **unbiased estimate** of $E_{out}(h^-)$. What about its **variance**?

Model Assessment - Holdout Test Set - Variance

- Assuming the test observations are i.i.d. and treating h^- as fixed, the variance of $E_{test}(h^-)$ is:

$$\begin{aligned} Var[E_{test}(h^-)] &= Var\left[\frac{1}{a}\sum_{l=1}^{a} err(h^-(\mathbf{x}_l), y_l)\right] \\ &= \frac{1}{a^2}\sum_{l=1}^{a} Var\left[err(h^-(\mathbf{x}_l), y_l)\right] \\ &= \frac{s^2}{a}, \end{aligned}$$

where

$$s^2 = Var_{(\mathbf{x},y)\sim P}\left[err(h^-(\mathbf{x}), y)\right].$$

- As the number a of test observations increases, the variance of $E_{test}(h^-)$ decreases. Thus, $E_{test}(h^-)$ provides a more reliable estimate of $E_{out}(h^-)$.
- However, increasing a leaves fewer observations for training the model.

Model Assessment - Holdout Algorithm

1. Compile the available data into $\mathcal{D} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$.
2. Randomly select a observations from $\mathcal{D}$ to form the test set $\mathcal{D}_{test}$, and place the remaining $n - a$ observations into the training set $\mathcal{D}_{train}$.
3. Use $\mathcal{D}_{train}$ to fit a model h^-.
4. Assess the performance of h^- using $\mathcal{D}_{test}$:

$$E_{test}(h^-) = \frac{1}{a} \sum_{(\mathbf{x}_l, y_l) \in \mathcal{D}_{test}} err(h^-(\mathbf{x}_l), y_l).$$

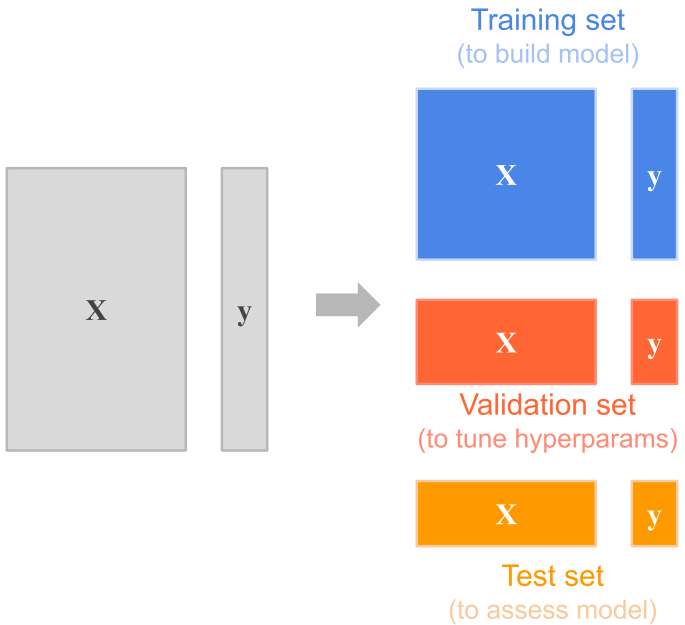
5. $E_{test}(h^-)$ provides an estimate of $E_{out}(h^-)$.

Refitting for Deployment

After assessment, the same model specification may be refitted using the entire dataset $\mathcal{D}$ to obtain a deployment model $\hat{h}$.

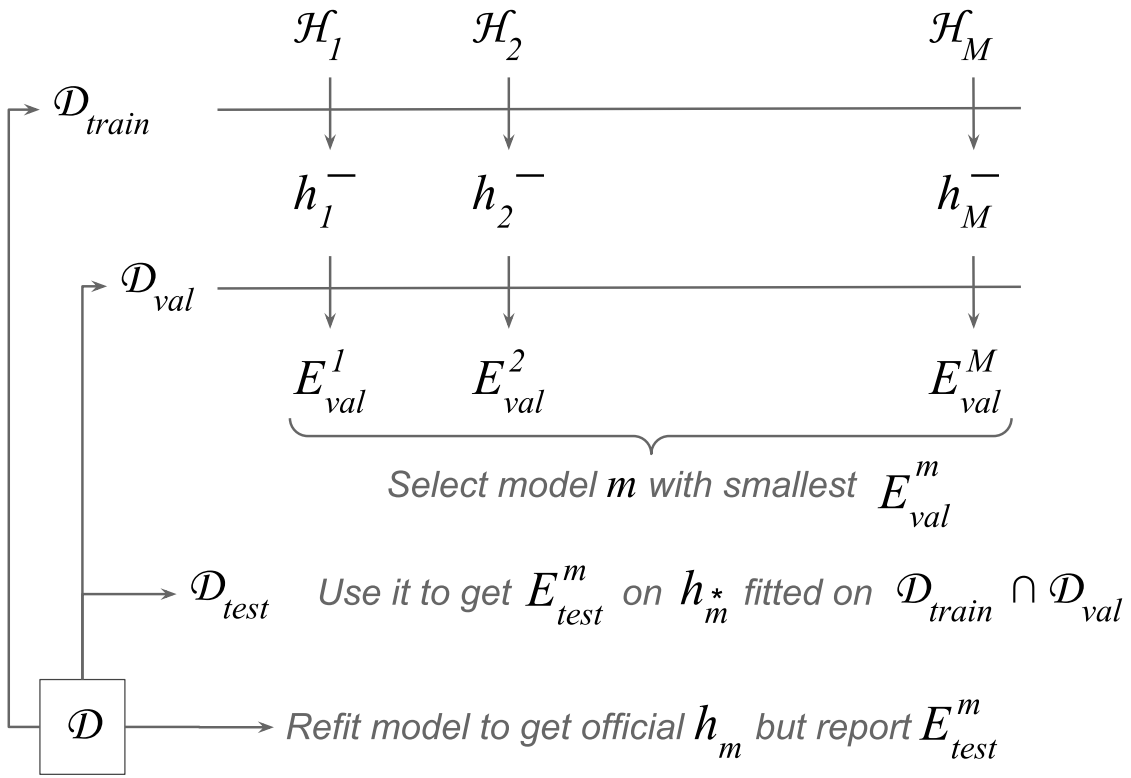
Note that $E_{test}(h^-)$ estimates the out-of-sample performance of h^-, not strictly that of the refitted model $\hat{h}$.

Model Selection - Three-Way Holdout Method



- The test set $\mathcal{D}_{test}$ should be **reserved until the end** of the learning process to estimate the out-of-sample error of the selected model.
- We do not use $\mathcal{D}_{test}$ to make any **learning decisions**, such as choosing hyperparameters or selecting among candidate models.
- Therefore, we introduce a validation set $\mathcal{D}_{val}$ for the **final-selection** of a model from a set of finalist models.

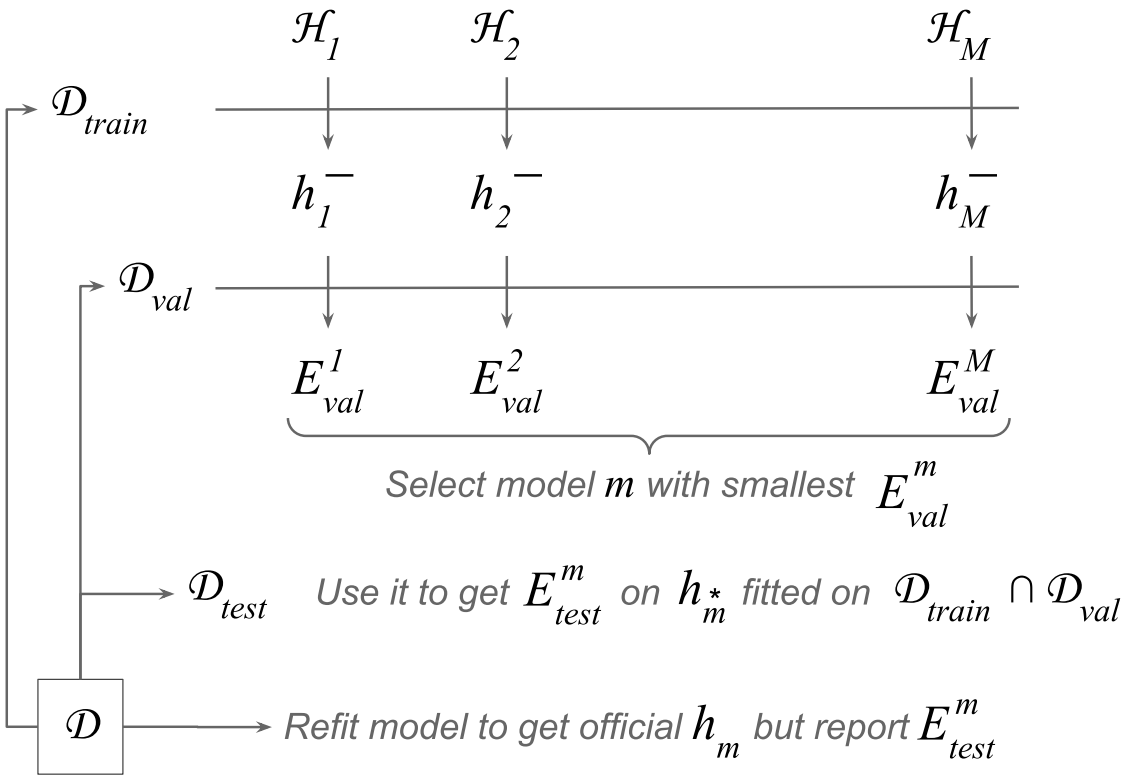
Model Selection



- $\mathcal{H}_m$ denotes the m-th hypothesis class, and h_m^- is the finalist model trained using $\mathcal{D}_{train}$.
- For each finalist h_m^-, we use $\mathcal{D}_{val}$ to compute the validation error

$$E_{val}^m = E_{val}(h_m^-).$$

Model Selection

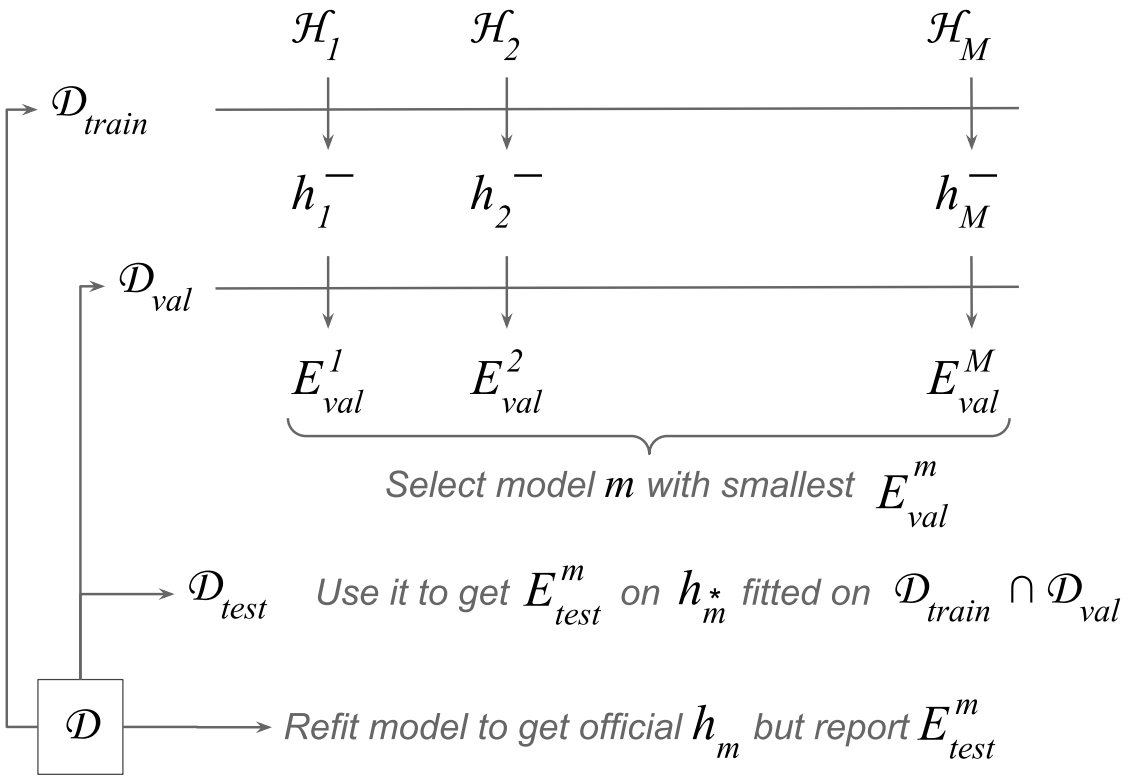


- Select the hypothesis class with the smallest validation error:

$$m^* = \arg\min_m E_{val}^m.$$

- A new model $h_{m^*}^*$ is then fitted using $\mathcal{D}_{train} \cup \mathcal{D}_{val}$.

Model Selection

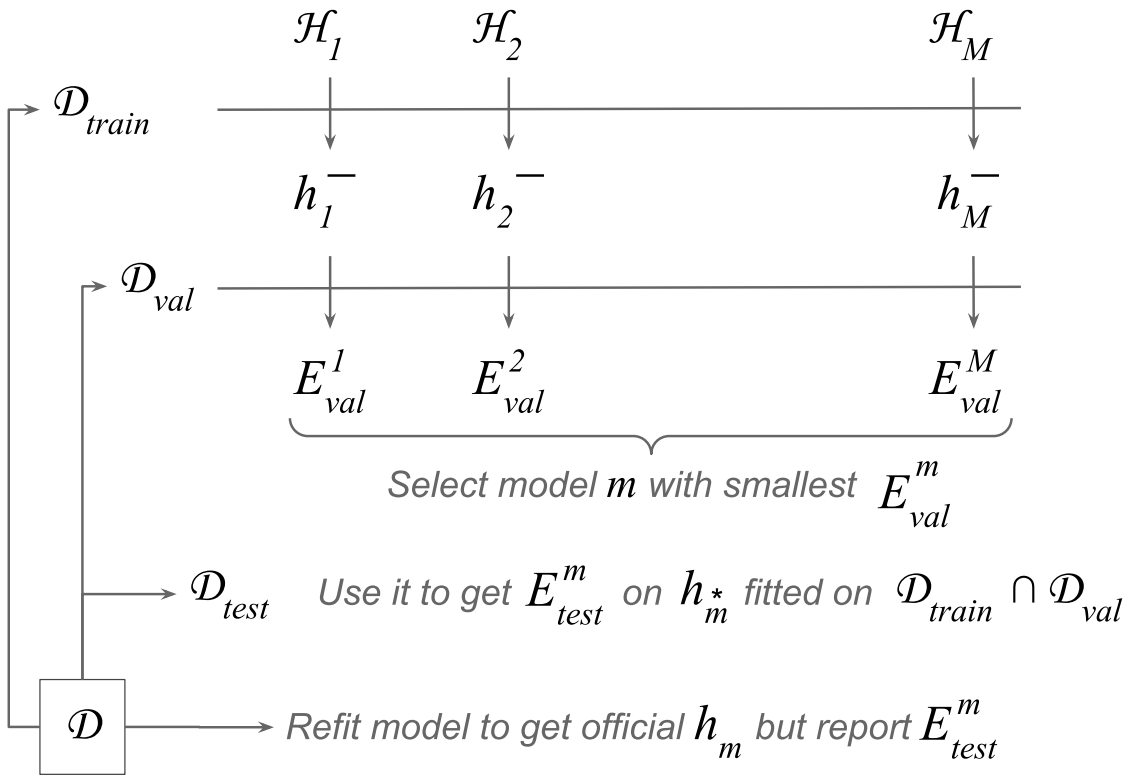


- The performance of $h^*_{m^*}$ is assessed using $\mathcal{D}_{test}$:

$$E^{m^*}_{test} = E_{test}(h^*_{m^*}).$$

- Finally, the "official" model is fitted on the entire dataset $\mathcal{D}$ using the selected hypothesis class $\mathcal{H}_{m^*}$.

Model Selection



- Choosing $m^* = \arg\min_m E_{val}^m$ is itself a **learning decision**.
- Therefore, $E_{val}^{m^*} = \min_m E_{val}^m$ tends to be an **optimistically biased estimate** of out-of-sample performance.
- Larger validation and test sets provide more reliable estimates of out-of-sample performance, but leave fewer examples for training.

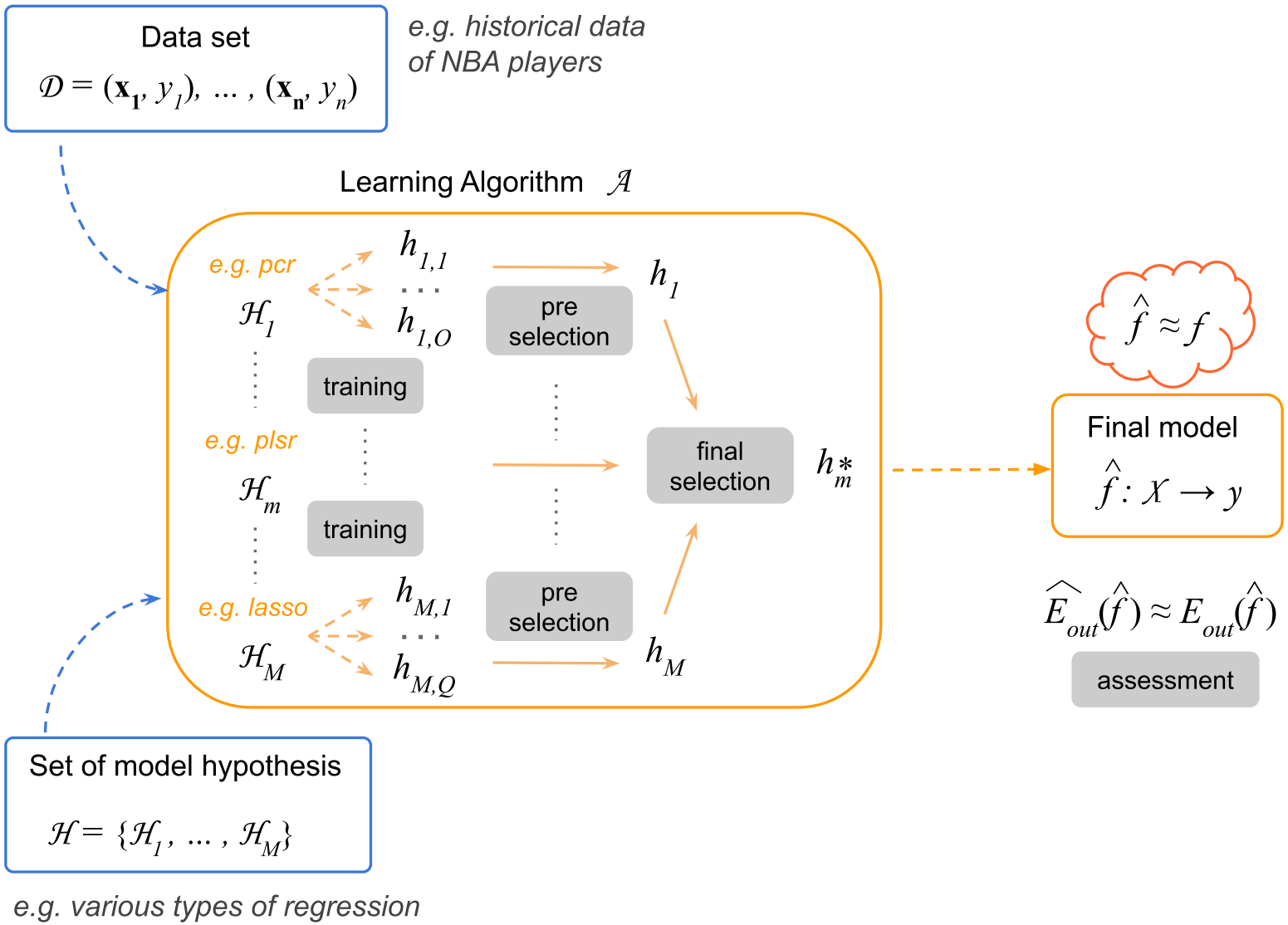
Model Training

- For each hypothesis class $\mathcal{H}_m$, we need to consider a set of candidate models corresponding to different hyperparameter values.
- For example, suppose $\mathcal{H}_m$ is ridge regression. We may consider candidate models

$$h_{\lambda_1,m}, h_{\lambda_2,m}, \ldots, h_{\lambda_L,m},$$

  corresponding to different values of the regularization parameter λ.
- If we use the same $\mathcal{D}_{train}$ both to fit the candidate models and to choose the one with the smallest training error, the selection may overfit to $\mathcal{D}_{train}$.
- We could use $\mathcal{D}_{val}$ for this pre-selection, but then $\mathcal{D}_{val}$ would no longer be **fresh** for the final-selection phase.
- Instead, we perform the pre-selection by repeatedly creating training and evaluation subsets from $\mathcal{D}_{train}$.
- $\longrightarrow$ Resampling methods.

Learning Phases



Resampling Approaches - General Sampling Blueprint

- We have a limited amount of training data $\mathcal{D}_{train}$.
- We want to use $\mathcal{D}_{train}$ for both:
  - fitting candidate models, and
  - evaluating them during the pre-selection phase.
- Resampling creates multiple training–evaluation splits from $\mathcal{D}_{train}$.
- At iteration b:
  1. Use $\mathcal{D}_{train-b}$ to fit a candidate model h_b.
  2. Use $\mathcal{D}_{eval-b}$ to evaluate h_b:

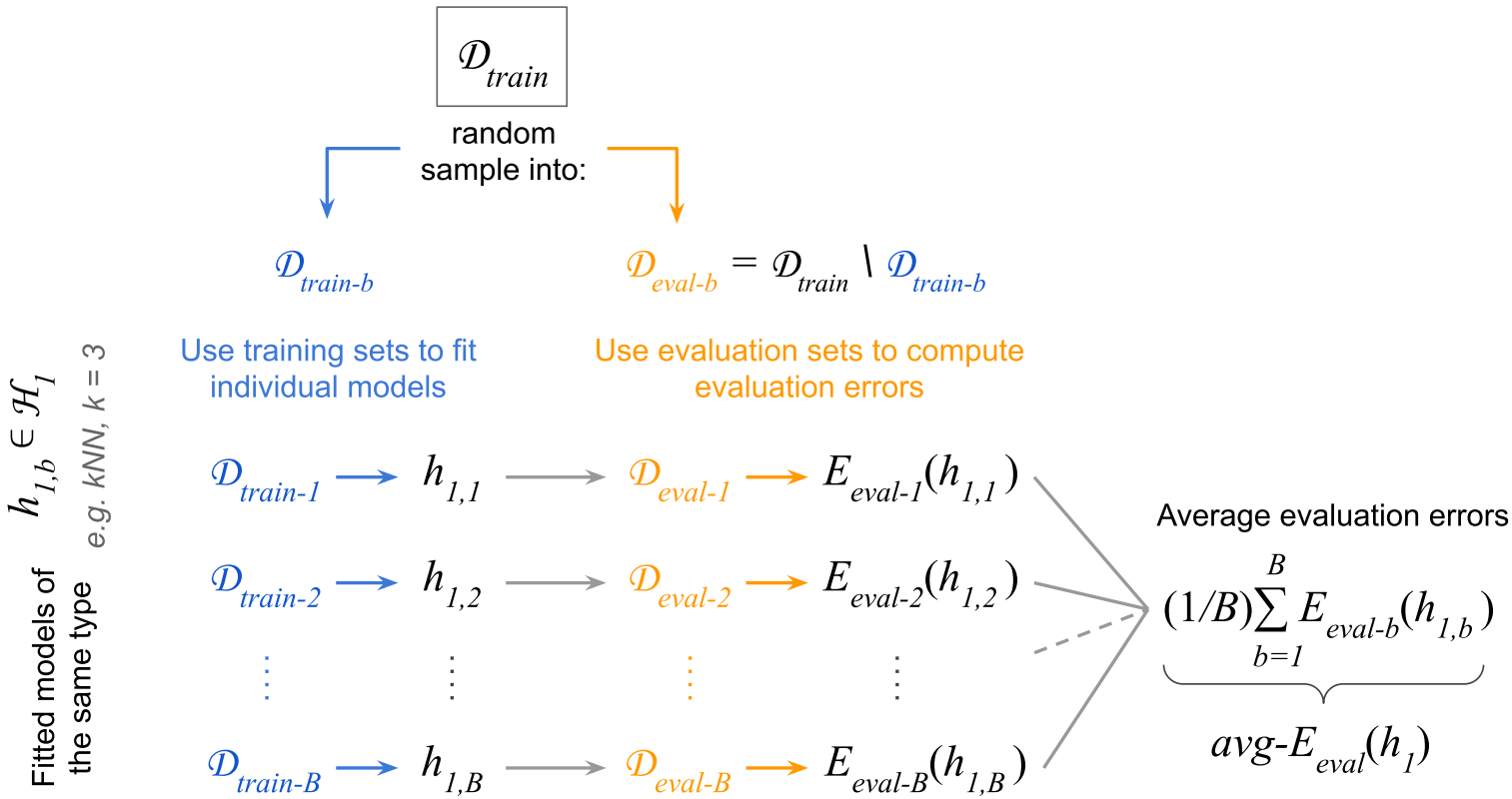
$$E_{eval-b} = \frac{1}{|\mathcal{D}_{eval-b}|} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{eval-b}} err(h_b(\mathbf{x}_i), y_i).$$

- Repeat the procedure over B training–evaluation splits and average the evaluation errors:

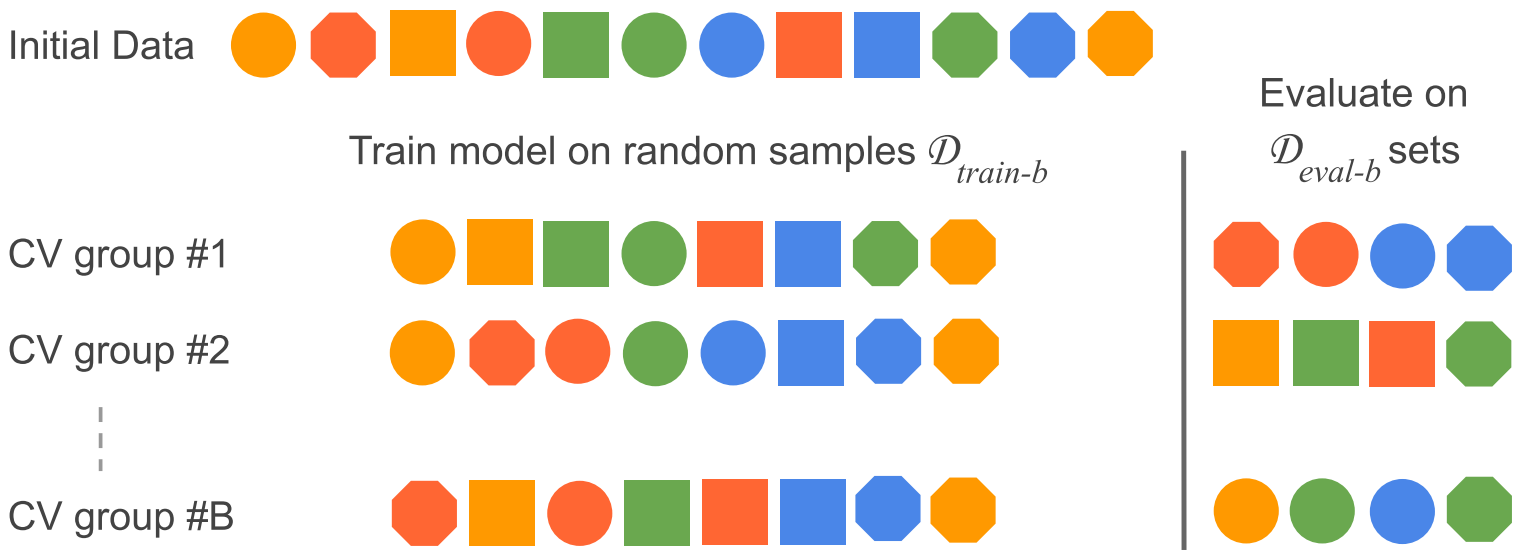
$$E_{eval} = \frac{1}{B} \sum_{b=1}^{B} E_{eval-b}.$$

- E_{eval} provides an estimate of the generalization performance of the candidate learning procedure.

Resampling Approaches



Monte Carlo Cross Validation

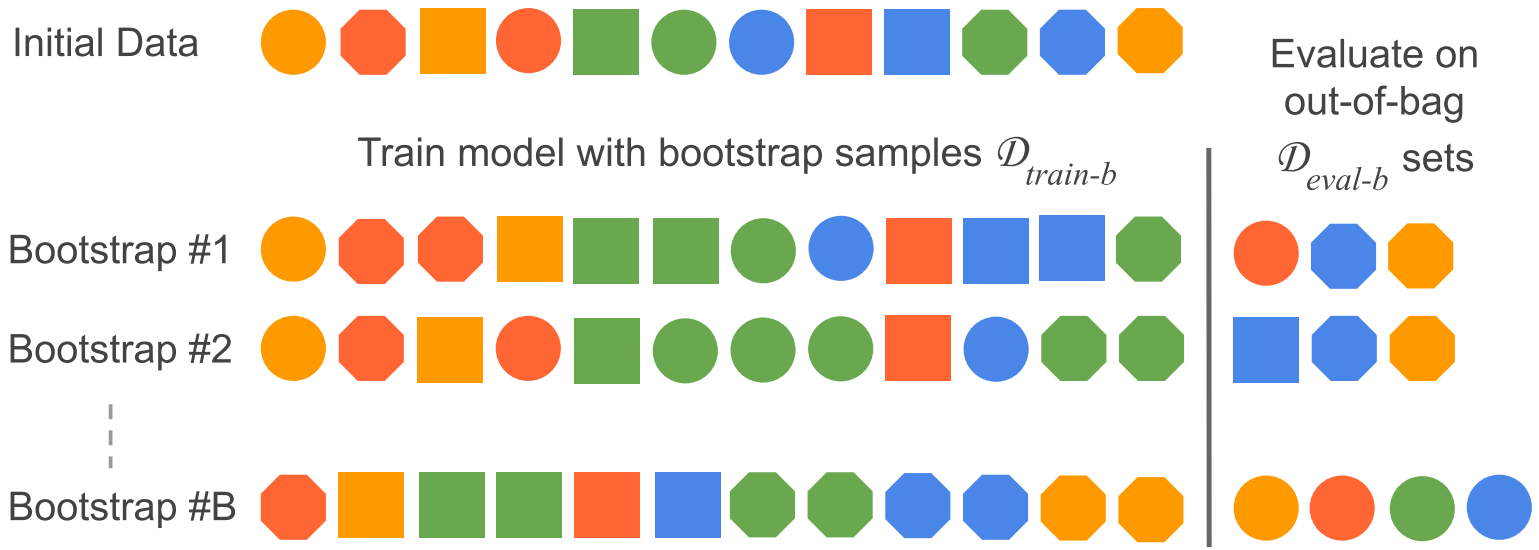


Monte Carlo Cross Validation - Repeated Holdout

1. Compile training dataset $\mathcal{D}_{train} = (\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)$
2. Repeat the Holdout method B times (e.g., $B = 50, 100$, or larger depending on computational cost).
   - Generate $\mathcal{D}_{train-b}$ by sampling $n - r$ elements without replacement from $\mathcal{D}_{train}$.
   - Generate h_b^- by fitting a model to $\mathcal{D}_{train-b}$.
   - Compute $E_{eval-b} = \frac{1}{r} \sum_i err(h_b^-(\mathbf{x}_i), y_i)$; where $(\mathbf{x}_i, y_i) \in \mathcal{D}_{eval-b}$.
3. Obtain an overall value for E_{eval} by averaging the E_{eval-b} values:

$$E_{eval} = \frac{1}{B} \sum_{b=1}^{B} E_{eval-b}$$

Bootstrap Method



- A bootstrap sample is obtained by sampling from the data **with replacement**.
- A bootstrap sample has the same size n as $\mathcal{D}_{train}$.
- Because sampling is with replacement, some observations may appear multiple times, while others may not be selected at all.
- The unselected observations are called **out-of-bag (OOB)** observations.

Bootstrap Algorithm

1. Compile the training dataset

$$\mathcal{D}_{train} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}.$$

2. Repeat the following steps for $b = 1, 2, \ldots, B$:
   - Generate a bootstrap training set $\mathcal{D}_{train-b}$ by sampling n observations **with replacement** from $\mathcal{D}_{train}$.
   - Fit the model on $\mathcal{D}_{train-b}$ to obtain h_b^-.
   - Let $\mathcal{D}_{eval-b}$ contain the **out-of-bag observations**, i.e., observations from $\mathcal{D}_{train}$ that were not selected in $\mathcal{D}_{train-b}$.
   - Compute the OOB evaluation error:

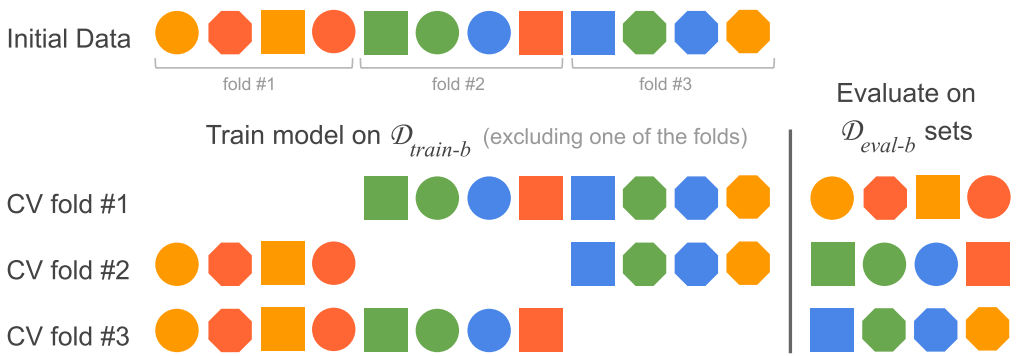
$$E_{eval-b} = \frac{1}{r_b} \sum_{(\mathbf{x}_i, y_i) \in \mathcal{D}_{eval-b}} err(h_b^-(\mathbf{x}_i), y_i),$$

   where $r_b = |\mathcal{D}_{eval-b}|$.

3. Average the evaluation errors over the B bootstrap samples:

$$E_{eval}^{bootstrap} = \frac{1}{B} \sum_{b=1}^{B} E_{eval-b}.$$

K-Fold Cross Validation



- Split $\mathcal{D}_{train}$ into K disjoint subsets of approximately equal size. Each subset is called a **fold**.
- For each iteration $b = 1, 2, \ldots, K$:
  - Use the b-th fold as the evaluation set $\mathcal{D}_{eval-b}$.
  - Merge the remaining $K - 1$ folds to form the training set $\mathcal{D}_{train-b}$.
  - Fit a model h_b^- on $\mathcal{D}_{train-b}$ and compute its evaluation error E_{eval-b} on $\mathcal{D}_{eval-b}$.
- The cross-validation error is obtained by averaging the evaluation errors over the K folds:

$$E_{cv} = \frac{1}{K} \sum_{b=1}^{K} E_{eval-b}.$$

Leave-One-Out Cross Validation (LOOCV)

Leave-One-Out Cross Validation is a special case of K-fold cross validation in which each fold contains a single data point, i.e., $K = n$.

1. Compile the training dataset

$$\mathcal{D}_{train} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}.$$

2. For $i = 1, 2, \ldots, n$:
   - Remove the i-th observation to form

$$\mathcal{D}_{train-i} = \mathcal{D}_{train} \setminus \{(\mathbf{x}_i, y_i)\}.$$

   - Fit the model on $\mathcal{D}_{train-i}$ to obtain h_i^-.
   - Use the held-out observation $(\mathbf{x}_i, y_i)$ to compute

$$E_{eval-i} = err(h_i^-(\mathbf{x}_i), y_i).$$

3. Obtain the LOOCV error by averaging the n held-out errors:

$$E_{cv} = \frac{1}{n} \sum_{i=1}^{n} E_{eval-i}.$$

Example - Tuning the Parameter λ of Ridge Regression

The ridge regression coefficients are given by:

$$\mathbf{w} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}$$

where $\lambda \geq 0$ is a tuning parameter (i.e., a hyperparameter).

- How can we choose λ to obtain the finalist model within the ridge regression hypothesis class?

Example - Tuning the Parameter λ of Ridge Regression

1. Split $\mathcal{D}_{train} = \{(\mathbf{x}_1, y_1), \ldots, (\mathbf{x}_n, y_n)\}$ into K folds: $\mathcal{D}_{train} = \bigcup_{b=1}^{K} \mathcal{D}_{fold-b}$.
2. For each $b = 1, 2, \ldots, K$, define

$$\mathcal{D}_{eval-b} = \mathcal{D}_{fold-b}, \qquad \mathcal{D}_{train-b} = \mathcal{D}_{train} \setminus \mathcal{D}_{fold-b}.$$

3. For each candidate value λ_l, $l = 1, 2, \ldots, L$:
   1. For $b = 1, 2, \ldots, K$:
      - Fit a ridge regression model $h_{l,b}$ with λ_l on $\mathcal{D}_{train-b}$.
      - Compute $E_{eval-b}(h_{l,b})$ on $\mathcal{D}_{eval-b}$.
   2. Compute the cross-validation error:

$$E_{cv-l} = \frac{1}{K} \sum_{b=1}^{K} E_{eval-b}(h_{l,b}).$$

4. Choose $\lambda^* = \arg\min_{\lambda_l} E_{cv-l}$.
5. Refit ridge regression on the entire $\mathcal{D}_{train}$ using λ^* to obtain the finalist model:

$$\mathbf{w}^* = (\mathbf{X}^\top \mathbf{X} + \lambda^* \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}.$$